

3 Chapter 3 Machine Learning Workflows

Machine learning workflows involve the sequence of steps used to prepare data, train models, and evaluate performance. In practice, successful machine learning applications often depend as much on data preparation and feature engineering as on the learning algorithm itself.

In this chapter, we introduce common workflows used in machine learning, including feature engineering, training and testing procedures, pipelines, and methods for improving model generalization.

3.1 Feature Engineering

Feature engineering is the process of transforming raw data into informative inputs that can be used by machine learning algorithms. Carefully designed features can significantly improve model performance and help capture important physical relationships in engineering data.

3.1.1 Time-Series Features

Many engineering systems generate data that varies over time, such as sensor measurements, vibration signals, or telemetry data. Common time-series features include moving averages, lagged values, rolling statistics, derivatives, and frequency-based quantities extracted from signal analysis techniques such as the Fourier transform.

3.1.2 Statistical Features

Statistical features summarize important properties of a dataset or signal using descriptive measures. Common examples include the mean, variance, standard deviation, minimum, maximum, skewness, and kurtosis, which can help characterize trends, variability, and abnormal behavior in engineering systems.

3.2 Training and Testing Data

Training a predictive model on a dataset and then testing it with the same data is a fundamental error in methodology. Such a model could simply memorize the labels of the training samples, achieving perfect performance during training but failing to make any useful predictions on new, unseen data. This phenomenon is known as overfitting. To prevent this, it is standard practice in supervised machine learning to reserve a portion of the available data as a test set, denoted as $\mathbf{X}_{\text{test}}$ and $\mathbf{y}_{\text{test}}$.

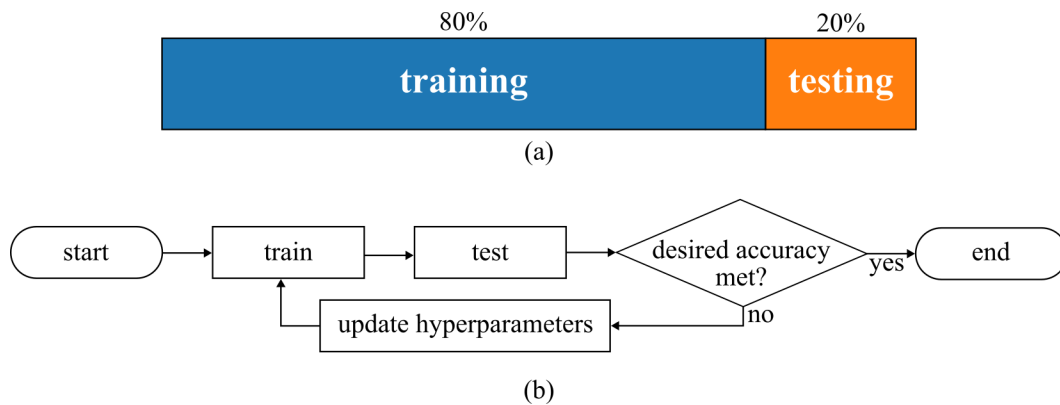


Figure 3.1: Splitting data up into training and testing subsets.

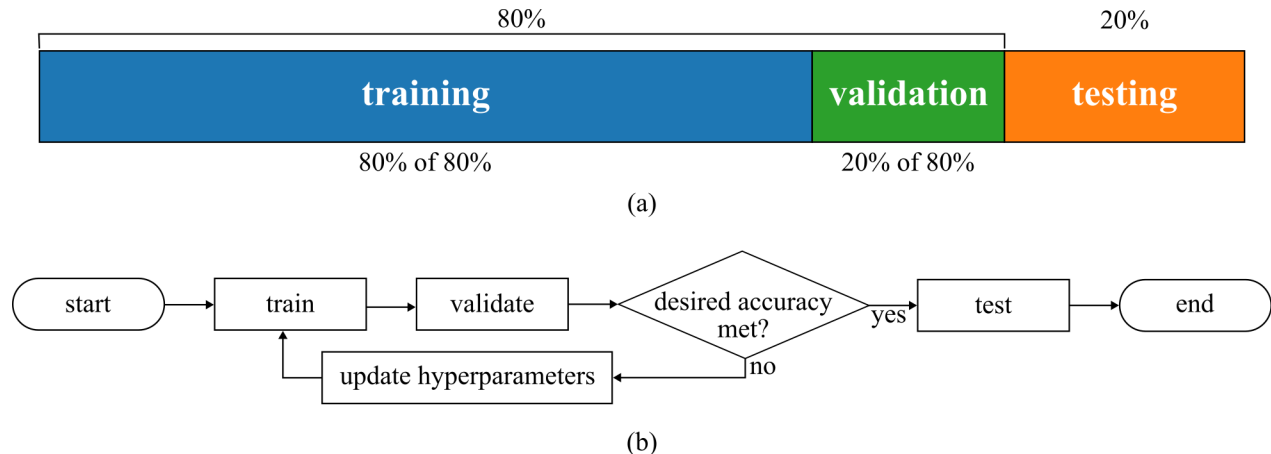


Figure 3.2: Splitting data up into training, validation, and testing subsets.

The function `sklearn.model_selection.train_test_split` in scikit-learn randomly divides arrays or matrices into training and testing subsets.

3.3 Pipelines

Pipelines in scikit-learn streamline the process by sequentially applying a list of transformations followed by a final estimator to a dataset. Employing pipelines allows for the integration of multiple processing steps, which can then be cross-validated together while experimenting with various parameters. Fig. 3.3 illustrates a typical pipeline configuration in scikit-learn.

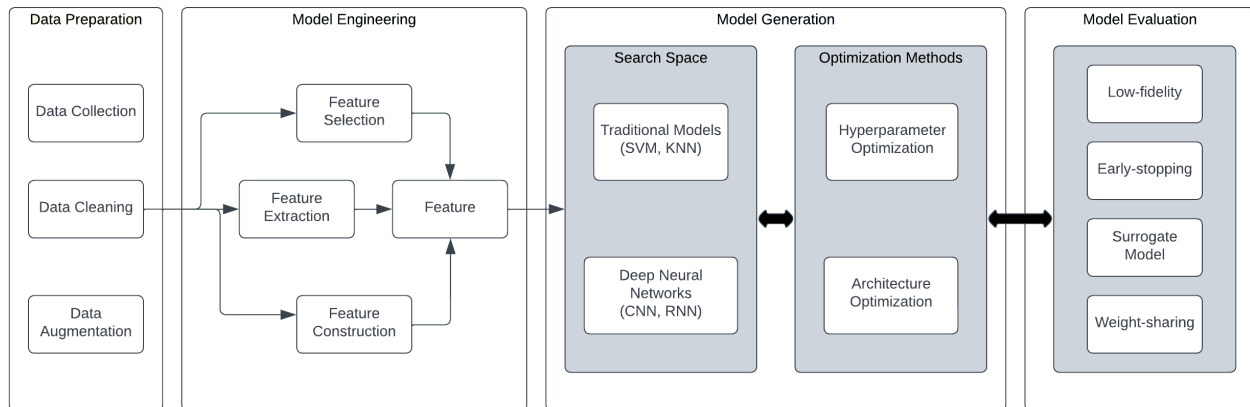


Figure 3.3: Pipeline setup for the automated deployment of pre-processing and modeling steps.^a

3.4 Learning Curves

Performing Polynomial Regression with a high degree typically allows for a closer fit to the training data compared to simple Linear Regression. For instance, Figure 3.4 demonstrates the application of a 30-degree polynomial model to a set of training data, contrasting it with both a linear model and a quadratic model (2nd-degree polynomial). Observe how the 30-degree polynomial model contorts to closely match the training instances.

- The linear model exhibits underfitting.
- The high-degree Polynomial Regression model drastically overfits the training data.
- Among these, the quadratic model is likely to generalize best.

In this instance, the appropriateness of the quadratic model is clear since the data was initially generated using such a model. However, in real-world scenarios where the underlying function of the data is unknown, determining the optimal complexity for your model can be challenging. How can you ascertain whether your model is overfitting or underfitting the data?

^aModified from PopovaZhuhadar, CC BY-SA 4.0 <<https://creativecommons.org/licenses/by-sa/4.0/>>, via Wikimedia Commons

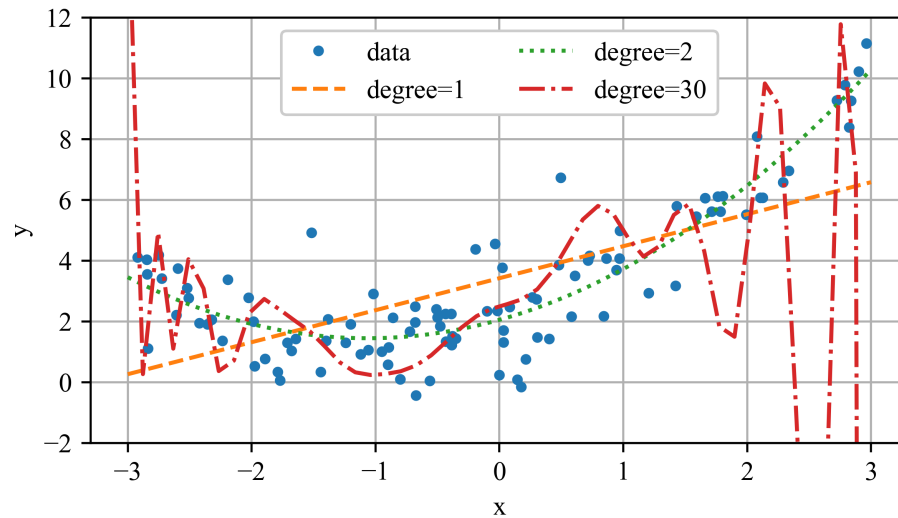


Figure 3.4: Polynomial regression showing underfitting (degree=1), a respectable model fit (degree=2), and overfitting (degree=30).

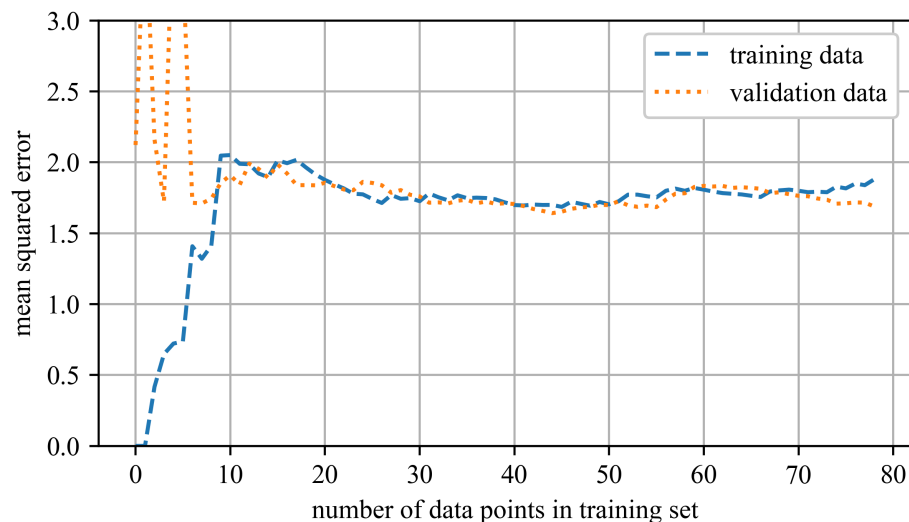


Figure 3.5: Learning curves for the underfitting linear model (degree=1) where underfitting is evident because both curves have plateaued; they are close together and relatively high.

Examining the model's behavior in figures 3.5 and 3.6 on the training set shows a clear trend. With only one or two samples the fit is perfect, so the error curve begins at zero. As additional observations are introduced the model can no longer capture every point exactly, partly because of measurement noise and partly because the underlying relationship is nonlinear, and consequently the training error rises. After a sufficient number of samples the curve flattens out; beyond this plateau adding further data neither markedly lowers nor raises the average error.

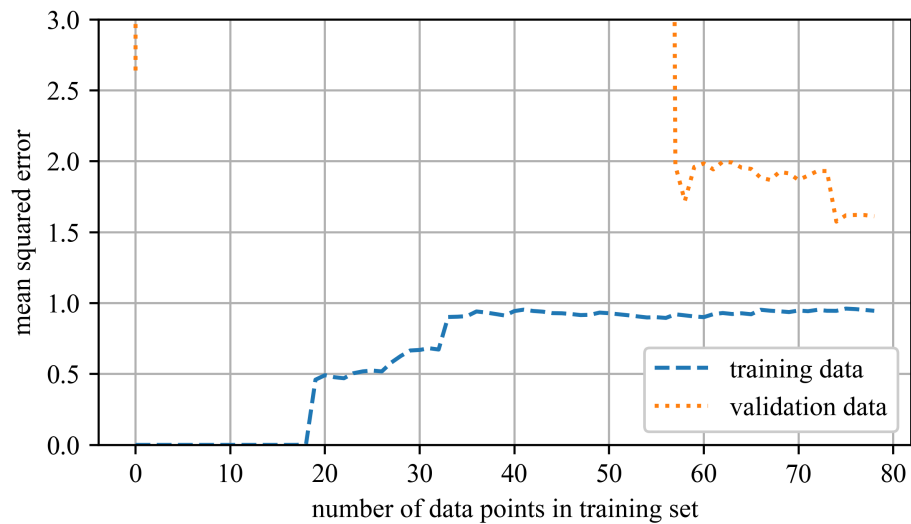


Figure 3.6: Learning curves for the overfitting polynomial model (degree=20) showing a significant gap between the curves, which indicates better performance on the training data than on the validation data.

Inspection of the validation curve in figure 3.6 tells a complementary story. With only a handful of training samples the model cannot generalize, so the validation error starts high. As more examples become available it learns progressively better patterns and the error falls. Eventually, though, the simple linear hypothesis is unable to capture the data's full complexity, causing the decline to flatten out; the validation curve plateaus at nearly the same level as the training curve. Their close proximity and uniformly large values are characteristic of an underfitting model.

Figure 3.6 displays the learning curves for a model fitted with a 20th-degree polynomial. The overall shape resembles the curves seen earlier, yet two key differences stand out. First, the error on the training set is far lower than that achieved by the Linear Regression model, demonstrating the polynomial's extra flexibility. Second, a pronounced gap separates the training and validation curves, which means the model performs much better on the data it has already seen; this divergence is the hallmark of overfitting. Expanding the training set could reduce the gap, but without additional regularization the risk of overfitting would remain.

Next, let's apply the code using a 2nd order polynomial, which accurately captures the essence of the data without underfitting or overfitting. These results are shown in Figure 3.7. Here it can be seen that the training and validation curves again meet after converging, showing a well-fit model.

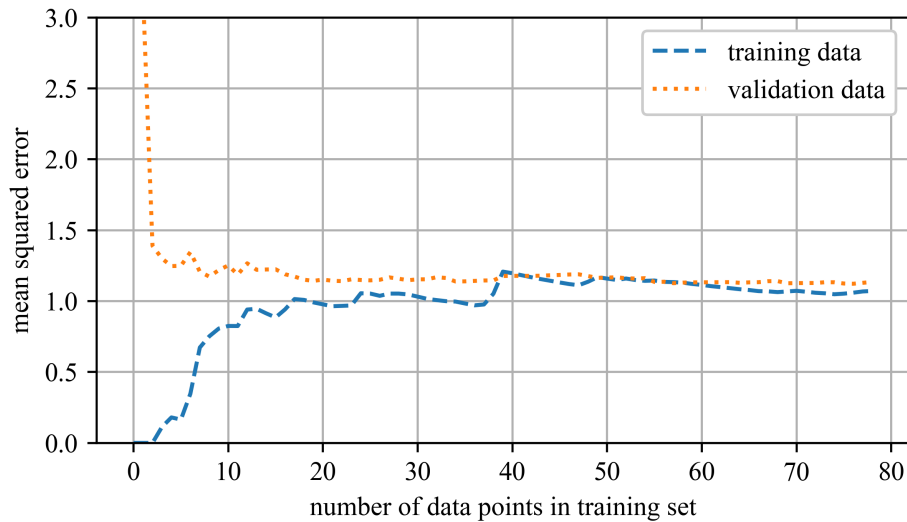


Figure 3.7: Learning curves for the optimal polynomial model (degree=2) showing well-aligned curves that plateau at a relatively low error level.

Example 3.1 Learning Curves

This example compares learning curves for linear and polynomial regression models. By plotting training and validation errors as the training set grows, it illustrates how model complexity impacts bias and variance, and helps identify underfitting and overfitting behavior.

3.5 Regularized Linear Models

Overfitting is frequently curbed by regularizing the model, meaning you introduce constraints that reduce its effective degrees of freedom and make it harder to memorize noise. In a polynomial setting you can achieve this simply by choosing a lower-degree polynomial, while in a linear model you typically add a penalty that discourages large coefficient values. This naturally raises the question: how is regularization expressed mathematically?

3.5.1 Ridge Regression

Ridge Regression modifies Linear Regression by adding a regularization term to the cost function, which compels the learning algorithm to fit the data while keeping the model weights as small as possible. The updated cost-function is

$$J(\theta) = \text{MSE}(\theta) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2. \quad (3.1)$$

The hyperparameter α dictates the degree of regularization. If $\alpha = 0$, Ridge Regression reduces to plain Linear Regression. If α is very high, the model weights are significantly shrunk, resulting in a nearly flat line through the mean of the data. Equation 3.1 outlines the cost function for Ridge Regression. The bias term θ_0 is not regularized, as indicated by the summation starting from $i = 1$.

NOTE

The regularization term is only included during the training phase. After training, the model's performance should be evaluated based on the unregularized performance measure.

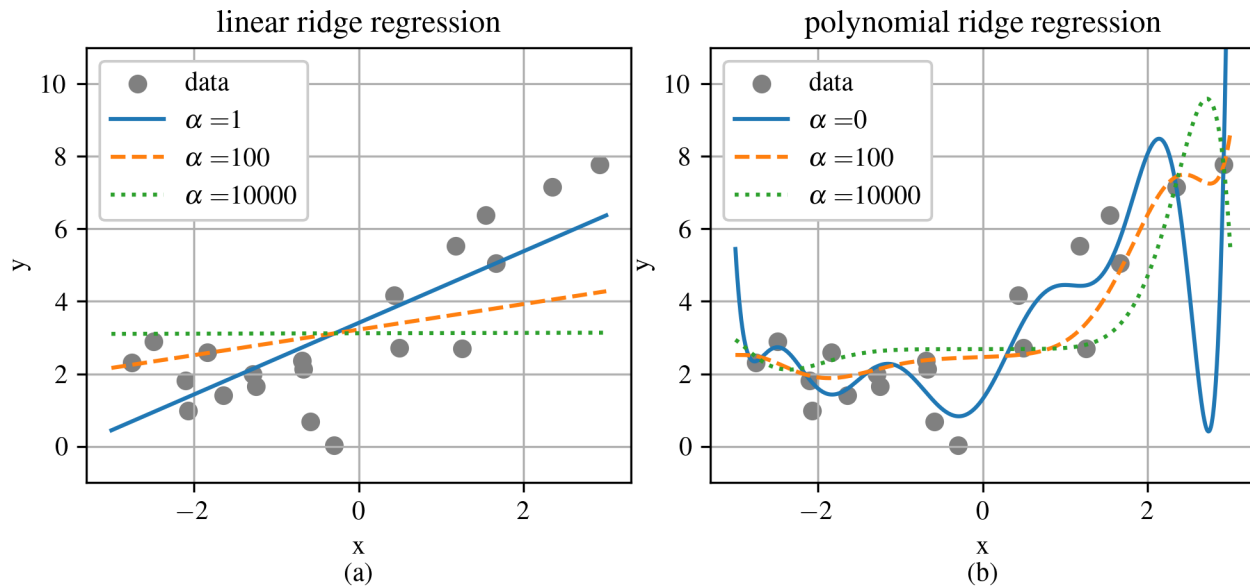


Figure 3.8: Visualization of Ridge Regression models with varying α values.

Figure 3.8 illustrates various Ridge models trained on linear data, showcasing different α value. In Figure 3.8(a), models are purely Ridge Regression, leading to linear predictions. In Figure 3.8(b), the data undergoes Polynomial Regression with Ridge regularization. To do this, the data is first expanded using `PolynomialFeatures(degree=10)`, then scaled with `StandardScaler`, and finally, Ridge models are applied to these transformed features.

Increasing α results in flatter (i.e., more reasonable and less extreme) predictions, thereby reducing the model's variance but increasing its bias. Ridge Regression can be performed using a closed-form solution or through Gradient Descent, similar to Linear Regression. The pros and cons for each method mirror those discussed previously. The closed-form solution, an extension of the normal equation, is given by:

$$\hat{\theta} = (X^T \cdot X + \alpha A)^{-1} \cdot X^T \cdot y \quad (3.2)$$

where α influences the regularization and A is an $n \times n$ identity matrix with the top-left value replaced by 0 to exclude the bias term. When using Gradient Descent, the derivative of the cost function guides the adjustment toward optimal model parameters.

Example 3.2 Ridge Regression

This example demonstrates Ridge Regression as a regularized alternative to linear regression. A high-degree polynomial model is fit to noisy data using a pipeline with Ridge regularization, showing how the regularization term reduces overfitting and stabilizes the model.

3.6 Early Stopping

An alternative approach to regularizing iterative learning methods (such as Gradient Descent) is to halt training as soon as the validation error hits its lowest point, a strategy known as early stopping. Figure 3.9 illustrates a complex model, specifically a high-degree Polynomial Regression model, being refined through Batch Gradient Descent. Over the course of training, as epochs progress, the model's prediction error (RMSE) on both the training and validation sets decreases. However, after some time, the validation error ceases to decline and begins to increase, signaling that the model is beginning to overfit the training data. By implementing early stopping, training is terminated the moment the validation error reaches its minimum. Geoffrey Hinton praised this method stating, “Early stopping (is) beautiful free lunch” due to its simplicity^a.

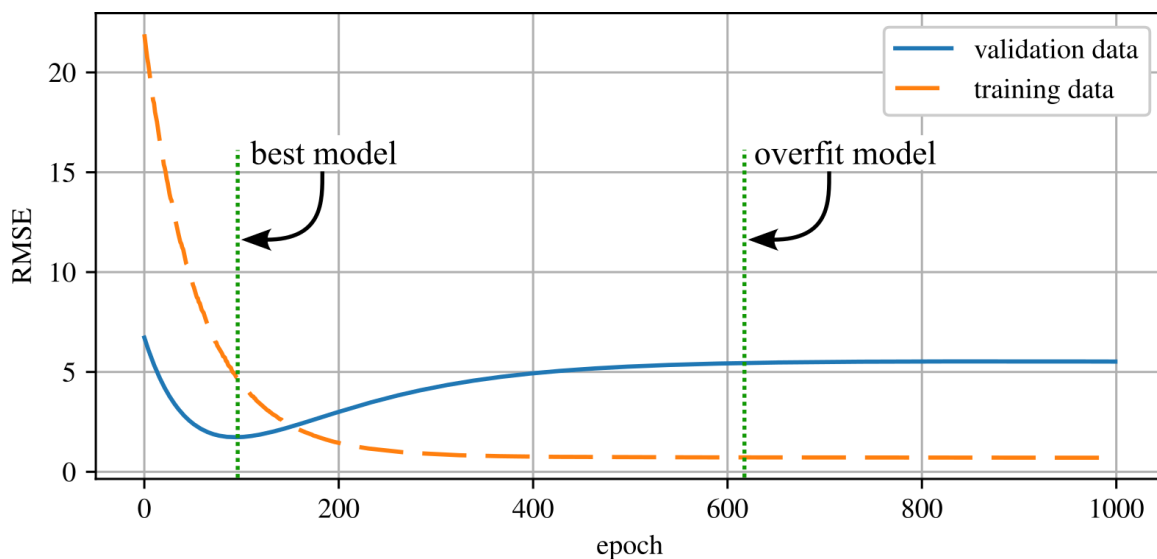


Figure 3.9: An example showing how early stopping can result in a better model as it does not allow for the over-fitting of the training set.

Example 3.3 Early Stopping

This example uses a high-degree polynomial model trained with stochastic gradient descent to demonstrate early stopping. By tracking training and validation errors across epochs, it highlights how halting training early can prevent overfitting and improve generalization.

^aPennington, Jeffrey, Richard Socher, and Christopher D. Manning. “Glove: Global vectors for word representation.” Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP). 2014.

3.7 Examples

Example 3.1

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 2.3
5  Learning curves
6  Machine Learning for Engineering Problem Solving
7  @author: Austin Downey
8  """
9
10 import IPython as IP
11 IP.get_ipython().run_line_magic('reset', '-sf')
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15 import sklearn as sk
16 from sklearn import linear_model
17 from sklearn import pipeline
18
19 plt.close('all')
20
21 %% build the data sets
22 np.random.seed(2) # 2 and 6 are pretty good
23 m = 100
24 X = 6 * np.random.rand(m,1) - 3
25 Y = 0.5 * X**2 + X + 2 + np.random.randn(m,1)
26 X_model = np.linspace(-3,3)
27
28 # plot the data
29 plt.figure()
30 plt.grid(True)
31 plt.scatter(X,Y)
32 plt.xlabel('x')
33 plt.ylabel('y')
34
35 %% generate learning curves for a linear model
36
37 # build the linear model in SK learn
38 model = sk.linear_model.LinearRegression()
39
40 # split the data into training and validation data sets
41 # Split arrays or matrices into random train and test subsets
42 X_train, X_val, y_train, y_val = sk.model_selection.train_test_split(X, Y, test_size=0.2)
43
44 train_errors, val_errors = [], []
45 for i in range(1, len(X_train)):
46     model.fit(X_train[:i], y_train[:i])
47     y_train_predict = model.predict(X_train[:i])
48     y_val_predict = model.predict(X_val)
49
50     # compute the error for the trained model
51     mse_train = sk.metrics.mean_squared_error(y_train[:i], y_train_predict)
52     train_errors.append(mse_train)
53
54     # compute the error for the validation model
55     mse_val = sk.metrics.mean_squared_error(y_val, y_val_predict)
56     val_errors.append(mse_val)
57
58 # predict model
59 y_model = model.predict(np.expand_dims(X_model,axis=1))
60
61 plt.figure('test model')
62 plt.scatter(X,Y,s=2, label='data')
63 plt.scatter(X_train[:i],y_train[:i], label='data in training set')
64 plt.scatter(X_val,y_val, marker='s', label='validation data')
65 plt.plot(X_model,y_model,'r--',label='model')
66 plt.xlabel('x')
67 plt.ylabel('y')

```

```

68     plt.legend(loc=2)
69     plt.grid(True)
70     plt.savefig('test_plots/linear_model_'+str(i))
71     plt.close('test model')
72
73     plt.figure()
74     plt.grid(True)
75     plt.plot(train_errors, "--", label="train")
76     plt.plot(val_errors, ":", label="val")
77     plt.xlabel('number of data points in training set')
78     plt.ylabel('mean squared error')
79     plt.legend(framealpha=1)
80     plt.ylim(0,6)
81     ### generate learning curves for a polynomial model
82
83     model = sk.pipeline.Pipeline((
84         ("poly_features", sk.preprocessing.PolynomialFeatures(degree=20, include_bias=False)),
85         ("lin_reg", sk.linear_model.LinearRegression()),
86     ))
87
88     # split the data into training and validation data sets
89     # Split arrays or matrices into random train and test subsets
90     X_train, X_val, y_train, y_val = sk.model_selection.train_test_split(X, Y, test_size=0.2)
91
92     train_errors = []
93     val_errors = []
94     for i in range(1, len(X_train)):
95         model.fit(X_train[:i], y_train[:i])
96         y_train_predict = model.predict(X_train[:i])
97         y_val_predict = model.predict(X_val)
98
99         # compute the error for the trained model
100        mse_train = sk.metrics.mean_squared_error(y_train[:i], y_train_predict)
101        train_errors.append(mse_train)
102
103        # compute the error for the validation model
104        mse_val = sk.metrics.mean_squared_error(y_val, y_val_predict)
105        val_errors.append(mse_val)
106
107        plt.figure('test model')
108        plt.scatter(X, Y, s=2, label='data')
109        plt.scatter(X_train[:i], y_train[:i], label='data in training set')
110        plt.scatter(X_val, y_val, marker='s', label='validation data')
111        y_model = model.predict(np.expand_dims(X_val, axis=1))
112        plt.plot(X_val, y_model, 'r--', label='model')
113        plt.xlabel('x')
114        plt.ylabel('y')
115        plt.legend(loc=2)
116        plt.grid(True)
117        plt.savefig('test_plots/polynominal_model_'+str(i))
118        plt.close('test model')
119
120    plt.figure()
121    plt.grid(True)
122    plt.plot(train_errors, "--", label="train")
123    plt.plot(val_errors, ":", label="val")
124    plt.xlabel('number of data points in training set')
125    plt.ylabel('mean squared error')
126    plt.legend(framealpha=1)
127    plt.ylim(0,6)
128
129
130
131

```

Example 3.2

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 2.4
5  Ridge Regression
6  Machine Learning for Engineering Problem Solving
7  @author: Austin Downey
8  """
9
10 import IPython as IP
11 IP.get_ipython().run_line_magic('reset', '-sf')
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15 import sklearn as sk
16 from sklearn import linear_model
17 from sklearn import pipeline
18
19 plt.close('all')
20
21 """ build the data sets
22 m = 20
23 X = 6 * np.random.rand(m, 1) - 3
24 Y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
25
26 X_model = np.linspace(-3,3,num=1000)
27 X_model = np.expand_dims(X_model,axis=1)
28
29
30 """ Perform Ridge Regression
31
32 # plot the data
33 plt.figure()
34 plt.grid(True)
35 plt.scatter(X,Y,color='gray')
36 plt.xlabel('x')
37 plt.ylabel('y')
38
39 # build and plot a linear model
40 model_linear = sk.linear_model.Ridge(alpha=100, solver="cholesky")
41 model_linear.fit(X, Y)
42 y_model_linear = model_linear.predict(X_model)
43 plt.plot(X_model,y_model_linear,'-',label='linear model')
44
45 # build and plot a polynomial model
46 model_poly = sk.pipeline.make_pipeline(sk.preprocessing.PolynomialFeatures(10),
47                                       sk.linear_model.Ridge(alpha=100, solver="cholesky"))
48 model_poly.fit(X, Y)
49 y_model_poly = model_poly.predict(X_model)
50 plt.plot(X_model,y_model_poly,'-',label='polynomial model')
51
52 plt.legend()
53
54
55
56
57
58

```

Example 3.3

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3  """
4  Example 2.5
5  Early Stopping
6  Machine Learning for Engineering Problem Solving
7  @author: Austin Downey
8  """
9
10 import IPython as IP
11 IP.get_ipython().run_line_magic('reset', '-sf')
12
13 import numpy as np
14 import matplotlib.pyplot as plt
15 import sklearn as sk
16
17 plt.close('all')
18
19 %% build the data sets
20
21 # use 6 to help give a smooth curve that makes the case for early stopping
22 np.random.seed(6)
23
24
25 m = 20
26 X = 6 * np.random.rand(m, 1) - 3
27 Y = 0.5 * X**2 + X + 2 + np.random.randn(m, 1)
28
29 # plot the data
30 plt.figure()
31 plt.grid(True)
32 plt.scatter(X,Y,color='gray')
33 plt.xlabel('x')
34 plt.ylabel('y')
35
36 X_model = np.linspace(-3,3,num=1000)
37 X_model = np.expand_dims(X_model,axis=1)
38
39 X_train, X_val, y_train, y_val = sk.model_selection.train_test_split(X, Y, test_size=0.2)
40
41 %% perform early stopping
42
43
44 # prepare the data
45 poly_scaler = sk.pipeline.Pipeline([("poly_features", sk.preprocessing.PolynomialFeatures(
46     degree=90, include_bias=False)), ("std_scaler", sk.preprocessing.StandardScaler())])
47 X_train_poly_scaled = poly_scaler.fit_transform(X_train)
48 X_val_poly_scaled = poly_scaler.fit_transform(X_val)
49
50 # set up the model, not that by setting max_iter=1 it will only train one epoch
51 model = sk.linear_model.SGDRegressor(max_iter=1, tol=0, learning_rate="constant"
52     ,eta0=0.0005,penalty=None,warm_start=True)
53
54
55 # Train the model in a loop to build the data set to investigate the benefit of early
56 # stopping
57 val_errors = []
58 train_errors = []
59 for epoch in range(1000):
60     model.fit(X_train_poly_scaled, y_train.ravel()) # continues where it left off
61     y_val_predict = model.predict(X_val_poly_scaled) # Predict the target values
62     y_train_predict = model.predict(X_train_poly_scaled) # Predict the target values
63     val_error = sk.metrics.mean_squared_error(y_val, y_val_predict) # Calculate error
64     train_error = sk.metrics.mean_squared_error(y_train.ravel(), y_train_predict) #
65     Calculate error
66     val_errors.append(val_error)

```

```
65     train_errors.append(train_error)
66
67     # plot the early learning curves, you may have to plot this a few times to get
68     # a set of curves that shows strong results
69     plt.figure()
70     plt.grid(True)
71     plt.plot(val_errors, label='validation data')
72     plt.plot(train_errors, '--', label='training data')
73     plt.ylabel('RMSE')
74     plt.xlabel('epoch')
75     plt.legend()
76
77
78
79
80
81
```